

Matemáticas para la Inteligencia Artificial

Ejercicios Entregables

Asignatura: 02MIAR_04_A_2026-27 — Matemáticas para la Inteligencia Artificial

Grupo: Nuevo Grupo 8

Curso: 2026-27

Integrantes del grupo

#	Nombre
---	--------

1	Adelso Steve Araya Solorzano
---	------------------------------

2	Fabio Antonio Rodriguez Quesada
---	---------------------------------

```
In [1]: import numpy as np
import sympy as sp
from numpy.polynomial import Polynomial
import math
```

Ejercicio 1 — Cálculo del Determinante

Tal y como ya hemos visto en clase, la variedad de herramientas proporcionadas por el álgebra lineal son cruciales para desarrollar y fundamentar las bases de una variedad de técnicas relacionadas con el aprendizaje automático. Con ella, podemos describir el proceso de propagación hacia adelante en una red neuronal, identificar mínimos locales en funciones multivariables (crucial para el proceso de retropropagación) o la descripción y empleo de métodos de reducción de la dimensionalidad, como el análisis de componentes principales (PCA), entre muchas otras aplicaciones.

Cuando trabajamos en la práctica dentro de este ámbito, la cantidad de datos que manejamos puede ser muy grande, por lo que es especialmente importante emplear algoritmos eficientes y optimizados para reducir el coste computacional en la medida de lo posible. Por todo ello, el objetivo de este ejercicio es el de ilustrar las diferentes alternativas que pueden existir para realizar un proceso relacionado con el álgebra lineal y el impacto que puede tener cada variante en términos del coste computacional del mismo. En este caso en particular, y a modo de ilustración, nos centraremos en el cálculo del determinante de una matriz.

a) [1 punto] Implementa una función, `determinante_recursivo`, que obtenga el determinante de una matriz cuadrada utilizando la definición recursiva de Laplace.

```
In [2]: def eliminar_fila_columna(matriz, fila, columna):
    matriz_secundaria = matriz.copy()
    matriz_secundaria = np.delete(matriz_secundaria, fila, axis=0)
    matriz_secundaria = np.delete(matriz_secundaria, columna, axis=1)
    return matriz_secundaria

def determinante_recursivo(matriz):
    if len(matriz) == 1:
        return matriz[0][0]

    if len(matriz) == 2:
        return matriz[0][0] * matriz[1][1] - matriz[0][1] * matriz[1][0]

    determinante = 0
    for i in range(len(matriz)):
        signo = (-1) ** i
        pivote = matriz[0][i]
        submatriz = eliminar_fila_columna(matriz, 0, i)
        determinante += signo * pivote * determinante_recursivo(submatriz)
    return determinante

matriz = np.array([[4, 2, -1], [3, 5, 2], [-1, 0, 7]])
print(f"Determinante de la matriz: {determinante_recursivo(matriz)}")
```

Determinante de la matriz: 89

b) [0.5 puntos] Si A es una matriz cuadrada $n \times n$ y triangular (superior o inferior, es decir, con entradas nulas por debajo o por encima de la diagonal, respectivamente), ¿existe alguna forma de calcular de forma directa y sencilla su determinante? Justifíquese la respuesta.

R/

Sí. En una matriz triangular (superior o inferior), al expandir el determinante por la definición de Laplace, todos los cofactores que involucran elementos fuera de la diagonal principal se anulan, ya que dichos elementos son cero. Por lo tanto, el único término que sobrevive en cada expansión es el correspondiente al elemento de la diagonal, lo que lleva a que el determinante sea simplemente el producto de los elementos de la diagonal principal:

$$\det(A) = \prod_{i=1}^n a_{ii} = a_{11} \cdot a_{22} \cdots a_{nn}$$

Esto aplica tanto para matrices triangulares superiores como inferiores, y también para matrices diagonales como caso particular.

Ejemplo práctico:

```
In [3]: # Matriz triangular superior
A = np.array([[3, 5, 2],
              [0, 4, 1],
              [0, 0, 2]])

det_diagonal = np.prod(np.diag(A))
det_numpy    = np.linalg.det(A)

print(f"Producto de la diagonal: {det_diagonal}")
print(f"Determinante (numpy):    {det_numpy:.1f}")
```

Producto de la diagonal: 24
Determinante (numpy): 24.0

c) [0.5 puntos] Determinése de forma justificada cómo alteran el determinante de una matriz $n \times n$ las dos operaciones elementales siguientes:

- Intercambiar una fila (o columna) por otra fila (o columna).
- Sumar a una fila (o columna) otra fila (o columna) multiplicada por un escalar α .

R/

A) Intercambiar dos filas (o columnas) de una matriz multiplica su determinante por -1 . Esto se debe a que el determinante es una función alternada en sus filas y columnas: cualquier intercambio de dos filas o columnas, sean adyacentes o no, invierte el signo del determinante.

$$\det(A') = -\det(A)$$

B) Sumar a una fila (o columna) otra fila (o columna) multiplicada por un escalar α **no altera** el determinante. Esto se debe a la linealidad del determinante: el término adicional genera una matriz con dos filas (o columnas) proporcionales, cuyo determinante es cero, por lo que la suma no modifica el valor original.

$$\det(A') = \det(A)$$

Ejemplo práctico:

```
In [4]: A = np.array([[4, 2, -1], [3, 5, 2], [-1, 0, 7]], dtype=float)

# A) Intercambiar filas 0 y 1
A_intercambiada = A[[1, 0, 2], :]
print(f"det(A) = {np.linalg.det(A):.2f}")
print(f"det(A intercambiada)= {np.linalg.det(A_intercambiada):.2f}")
print()

# B) Sumar a fila 1 la fila 0 multiplicada por alpha=2
alpha = 2
A_sumada = A.copy()
A_sumada[1] = A_sumada[1] + alpha * A_sumada[0]
print(f"det(A) = {np.linalg.det(A):.2f}")
print(f"det(A sumada)= {np.linalg.det(A_sumada):.2f}")

det(A) = 89.00
det(A intercambiada)= -89.00

det(A) = 89.00
det(A sumada)= 89.00
```

d) [1 punto] Investiga sobre el método de eliminación de Gauss con pivoteo parcial e impleméntalo para escalonar una matriz (es decir, convertirla en una matriz triangular inferior) a partir de las operaciones elementales descritas en el apartado anterior.

```
In [5]: def eliminacion_gauss_pivoteo(matriz):
    A = matriz.copy().astype(float)
    n = len(A)

    for col in range(n - 1, -1, -1):
        # Pivoteo parcial: intercambiar con la fila de mayor valor absoluto
        fila_max = np.argmax(np.abs(A[:col + 1, col]))
        if fila_max != col:
            A[[col, fila_max]] = A[[fila_max, col]]
```

```

# Eliminación: hacer ceros por encima del pivote
for fila in range(col - 1, -1, -1):
    if A[col][col] == 0:
        continue
    multiplicador = A[fila][col] / A[col][col]
    A[fila] = A[fila] - multiplicador * A[col]

return A

# Ejemplo de uso
matriz = np.array([[4, 2, -1],
                  [3, 5, 2],
                  [-1, 0, 7]], dtype=float)

resultado = eliminacion_gauss_pivoteo(matriz)
print("Matriz triangular inferior:")
print(np.round(resultado, 5))

```

```

Matriz triangular inferior:
[[ 2.54286  0.      0.    ]
 [ 3.28571  5.      0.    ]
 [-1.      0.      7.    ]]

```

e) [0.5 puntos] ¿Cómo se podría calcular el determinante de una matriz haciendo beneficio de la estrategia anterior y del efecto de aplicar las operaciones elementales pertinentes? Implementa una nueva función, `determinante_gauss`, que calcule el determinante de una matriz utilizando eliminación gaussiana.

```

In [6]: def determinante_gauss(matriz):
        A = matriz.copy().astype(float)
        n = len(A)
        signo = 1

        # Rastrear intercambios de filas para ajustar el signo del determinante
        for col in range(n - 1, -1, -1):
            fila_max = np.argmax(np.abs(A[:col + 1, col]))
            if fila_max != col:
                signo *= -1

        # Usar la función del apartado d) para obtener la matriz triangular inferior
        A_triangular = eliminacion_gauss_pivoteo(matriz)

        # El determinante es el producto de la diagonal ajustado por el signo
        return signo * np.prod(np.diag(A_triangular))

# Verificación con la misma matriz de 1a)
matriz = np.array([[4, 2, -1],
                  [3, 5, 2],
                  [-1, 0, 7]], dtype=float)

print(f"Determinante (Gauss):      {determinante_gauss(matriz):.2f}")

```

```
print(f"Determinante (numpy):      {np.linalg.det(matriz):.2f}")
print(f"Determinante (recursivo): {determinante_recursivo(matriz)}")
```

```
Determinante (Gauss):      89.00
Determinante (numpy):      89.00
Determinante (recursivo): 89.0
```

f) [0.5 puntos] Obtén la complejidad computacional asociada al cálculo del determinante con la definición recursiva y con el método de eliminación de Gauss con pivoteo parcial.

R/

Definición recursiva de Laplace — $O(n!)$

En cada llamada recursiva, se expande por n cofactores, y cada uno genera una submatriz de tamaño $(n - 1) \times (n - 1)$. El número total de operaciones sigue la recurrencia:

$$T(n) = n \cdot T(n - 1) \implies T(n) = O(n!)$$

Esto lo hace inviable para matrices grandes, ya que el coste crece factorialmente.

Eliminación de Gauss con pivoteo parcial — $O(n^3)$

El algoritmo realiza n iteraciones externas (una por columna). En cada iteración se busca el pivote en $O(n)$ y se eliminan los elementos de n filas, cada una con n operaciones. Por tanto:

$$T(n) = n \cdot n \cdot n = O(n^3)$$

Esto representa una mejora drástica frente al método recursivo, haciendo viable el cálculo para matrices de gran tamaño.

g) [1 punto] Utilizando `numpy.random.rand`, genera matrices cuadradas aleatorias de la forma $A_n \in \mathbb{R}^{n \times n}$, para $2 \leq n \leq 10$, y confecciona una tabla comparativa del tiempo de ejecución asociado a cada una de las variantes siguientes, interpretando los resultados:

- Utilizando `determinante_recursivo`.
- Empleando `determinante_gauss`.
- Haciendo uso de la función preprogramada `numpy.linalg.det`.

```
In [7]: import time

print(f"{'n':>4} | {'Recursivo (s)':>15} | {'Gauss (s)':>12} | {'NumPy (s)':>12}")
print("-" * 52)
```

```

for n in range(2, 11):
    A = np.random.rand(n, n)

    t0 = time.perf_counter()
    for _ in range(10): determinante_recursivo(A)
    t_recursivo = (time.perf_counter() - t0) / 10

    t0 = time.perf_counter()
    for _ in range(10): determinante_gauss(A)
    t_gauss = (time.perf_counter() - t0) / 10

    t0 = time.perf_counter()
    for _ in range(10): np.linalg.det(A)
    t_numpy = (time.perf_counter() - t0) / 10

    print(f"{n:>4} | {t_recursivo:>15.6f} | {t_gauss:>12.6f} | {t_numpy:>12.6f}")

```

n	Recursivo (s)	Gauss (s)	NumPy (s)
2	0.000001	0.000008	0.000002
3	0.000009	0.000012	0.000002
4	0.000039	0.000014	0.000001
5	0.000207	0.000021	0.000001
6	0.001269	0.000030	0.000002
7	0.009569	0.000046	0.000003
8	0.071181	0.000047	0.000003
9	0.637751	0.000066	0.000004
10	6.372181	0.000075	0.000007

Interpretación:

- **determinante_recursivo** crece factorialmente $O(n!)$, por lo que se vuelve significativamente más lento a medida que n aumenta.
- **determinante_gauss** crece de forma cúbica $O(n^3)$, siendo mucho más eficiente que el método recursivo para matrices grandes.
- **numpy.linalg.det** es el más rápido en todos los casos, ya que utiliza implementaciones optimizadas en C (LAPACK) con algoritmos altamente eficientes.

Esto confirma que la elección del algoritmo tiene un impacto drástico en el rendimiento computacional, especialmente al escalar el tamaño de los datos.

Ejercicio 2 — Descenso de Gradiente

En este ejercicio trabajaremos con el método de descenso de gradiente, el cual constituye otra herramienta crucial, en esta ocasión de la rama del cálculo, para el proceso de retropropagación asociado al entrenamiento de una red neuronal.

a) [1 punto] Prográtese en Python el método de descenso de gradiente para funciones de n variables. La función deberá tener como parámetros de entrada:

- El gradiente de la función que se desea minimizar ∇f (puede venir dada como otra función previamente implementada, `grad_f`, con entrada un vector, representando el punto donde se quiere calcular el gradiente, y salida otro vector, representando el gradiente de f en dicho punto).
- Un valor inicial $x_0 \in \mathbb{R}^n$ (almacenado en un vector de n componentes).
- El ratio de aprendizaje γ (que se asume constante para cada iteración).
- Un parámetro de tolerancia `tol` (con el que finalizar el proceso cuando $\|\nabla f(x)\|_2 < \text{tol}$).
- Un número máximo de iteraciones `maxit` (con el fin de evitar ejecuciones indefinidas en caso de divergencia o convergencia muy lenta).

La salida de la función deberá ser la aproximación del x que cumple $f'(x) \approx 0$, correspondiente a la última iteración realizada en el método.

```
In [8]: def descenso_gradiente(grad_f, x0, gamma, tol=1e-12, maxit=int(1e5)):
        x = np.array(x0, dtype=float)

        for _ in range(maxit):
            with np.errstate(over='ignore', invalid='ignore'):
                grad = np.array(grad_f(x), dtype=float)

            # Detener si el método diverge (valores no finitos)
            if not np.all(np.isfinite(grad)):
                return np.full_like(x, np.inf)

            if np.linalg.norm(grad) < tol:
                break

            x = x - gamma * grad

        return x

# Funciones auxiliares para los apartados b) y c)
def grad_f(x):
    # Gradiente de f(x) = 3x^4 + 4x^3 - 12x^2 + 7
    # f'(x) = 12x^3 + 12x^2 - 24x
    return np.array([12*x[0]**3 + 12*x[0]**2 - 24*x[0]])
```



```
def grad_g(x):
    # Gradiente de g(x,y) = x^2 + y^3 + 3xy + 1
    # dg/dx = 2x + 3y
    # dg/dy = 3y^2 + 3x
    return np.array([2*x[0] + 3*x[1], 3*x[1]**2 + 3*x[0]])
```

b) Sea la función $f: \mathbb{R} \rightarrow \mathbb{R}$ dada por

$$f(x) = 3x^4 + 4x^3 - 12x^2 + 7$$

b.i) [0.5 puntos] Aplica el método sobre $f(x)$ con $x_0 = 3$, $\gamma = 0.001$, `tol=1e-12`, `maxit=1e5`.

```
In [9]: # b.i) x0=3, gamma=0.001
resultado = descenso_gradiente(grad_f, x0=[3], gamma=0.001, tol=1e-12, maxit=
x = resultado[0]
print(f"Aproximación del mínimo: x = {x:.6f}")
print(f"f(x) = {3*x**4 + 4*x**3 - 12*x**2 + 7:.6f}")
```

Aproximación del mínimo: $x = 1.000000$
 $f(x) = 2.000000$

b.ii) [0.5 puntos] Aplica de nuevo el método sobre $f(x)$ con $x_0 = 3$, $\gamma = 0.01$, `tol=1e-12`, `maxit=1e5`.

```
In [10]: # b.ii) x0=3, gamma=0.01
resultado = descenso_gradiente(grad_f, x0=[3], gamma=0.01, tol=1e-12, maxit=
x = resultado[0]
print(f"Aproximación del mínimo: x = {x:.6f}")
print(f"f(x) = {3*x**4 + 4*x**3 - 12*x**2 + 7:.6f}")
```

Aproximación del mínimo: $x = -2.000000$
 $f(x) = -25.000000$

b.iii) [0.5 puntos] Contrasta e interpreta los dos resultados obtenidos en los apartados anteriores y compáralos con los mínimos locales obtenidos analíticamente. ¿Qué influencia puede llegar a tener la elección del ratio de aprendizaje γ ?

R/

Resultados obtenidos:

Apartado	γ	x aproximado	$f(x)$	Tipo
b.i	0.001	$x = 1$	2	Mínimo local
b.ii	0.01	$x = -2$	-25	Mínimo global

Contraste con el estudio analítico:

La función $f(x) = 3x^4 + 4x^3 - 12x^2 + 7$ tiene dos mínimos: uno local en $x = 1$ ($f = 2$) y uno global en $x = -2$ ($f = -25$). Ambas ejecuciones convergen a mínimos

válidos, pero a mínimos **distintos**, pese a partir del mismo punto inicial $x_0 = 3$.

Influencia del ratio de aprendizaje γ :

- Con $\gamma = 0.001$, los pasos son muy pequeños y el algoritmo desciende suavemente hacia el mínimo **más cercano** a la derecha ($x = 1$), quedando "atrapado" en el mínimo local.
- Con $\gamma = 0.01$, los pasos son mayores y permiten que el algoritmo "salte" la barrera del máximo local en $x = 0$, alcanzando el mínimo **global** en $x = -2$.

Esto demuestra que la elección de γ no solo afecta la velocidad de convergencia, sino también **a qué mínimo converge** el método. Un γ pequeño es más estable pero propenso a estancarse en mínimos locales; un γ mayor puede escapar de ellos, aunque con el riesgo de oscilar o divergir si es excesivamente grande.

b.iv) [0.5 puntos] Aplica nuevamente el método sobre $f(x)$ con $x_0 = 3$, $\gamma = 0.1$, `tol=1e-12`, `maxit=1e5`. Interpreta el resultado.

```
In [11]: # b.iv) x0=3, gamma=0.1
resultado = descenso_gradiente(grad_f, x0=[3], gamma=0.1, tol=1e-12, maxit=1e5)
x = resultado[0]

if np.isnan(x) or np.isinf(x):
    print("El método DIVERGE: el valor de x tiende a infinito (overflow).")
else:
    print(f"Aproximación del mínimo: x = {x:.6f}")
    print(f"f(x) = {3*x**4 + 4*x**3 - 12*x**2 + 7:.6f}")
```

El método DIVERGE: el valor de x tiende a infinito (overflow).

R/

Con $\gamma = 0.1$ el método **diverge**: el valor de x crece sin control hasta producir un desbordamiento numérico (`nan` /overflow).

Esto ocurre porque el paso $\gamma \cdot \nabla f(x)$ es demasiado grande. En $x_0 = 3$, el gradiente vale $f'(3) = 360$, por lo que la primera actualización es $x = 3 - 0.1 \cdot 360 = -33$. En ese nuevo punto el gradiente es aún mayor en magnitud, generando saltos cada vez más grandes que alejan al algoritmo del mínimo en lugar de acercarlo.

Este comportamiento confirma que un ratio de aprendizaje excesivo rompe la convergencia del descenso de gradiente, incluso partiendo del mismo punto inicial que en los apartados anteriores.

b.v) [0.5 puntos] Finalmente, aplica el método sobre $f(x)$ con $x_0 = 0$, $\gamma = 0.001$, `tol=1e-12`, `maxit=1e5`. Interpreta el resultado y compáralo con el estudio analítico de f . ¿Se trata de un resultado deseable? ¿Por qué? ¿A qué se debe este fenómeno?

```
In [12]: # b.v) x0=0, gamma=0.001
resultado = descenso_gradiente(grad_f, x0=[0], gamma=0.001, tol=1e-12, maxit
x = resultado[0]
print(f"Aproximación: x = {x:.6f}")
print(f"Gradiente en x: {grad_f(resultado)[0]:.6f}")
print(f"f(x) = {3*x**4 + 4*x**3 - 12*x**2 + 7:.6f}")
```

Aproximación: x = 0.000000

Gradiente en x: 0.000000

f(x) = 7.000000

R/

El método se queda estancado en $x = 0$ sin moverse. Esto **no es un resultado deseable**, porque $x = 0$ no es un mínimo sino un **máximo local** de la función.

¿A qué se debe este fenómeno?

El punto inicial $x_0 = 0$ es precisamente un punto crítico, donde el gradiente vale exactamente cero: $f'(0) = 0$. Como la actualización del descenso de gradiente es $x = x - \gamma \cdot f'(x)$, si $f'(x) = 0$ entonces x nunca cambia. El algoritmo cumple la condición de parada en la primera iteración y devuelve el mismo punto inicial, quedando atrapado en un máximo local.

Estudio analítico de f

$$f(x) = 3x^4 + 4x^3 - 12x^2 + 7$$

1) Primera derivada (puntos críticos):

$$f'(x) = 12x^3 + 12x^2 - 24x = 12x(x^2 + x - 2) = 12x(x + 2)(x - 1)$$

Igualando a cero, los puntos críticos son: $x = -2$, $x = 0$, $x = 1$.

2) Segunda derivada (clasificación):

$$f''(x) = 36x^2 + 24x - 24$$

x	$f''(x)$	Tipo	$f(x)$
-2	$36(4) - 48 - 24 = 72 > 0$	Mínimo local	-25
0	$-24 < 0$	Máximo local	7
1	$36 + 24 - 24 = 36 > 0$	Mínimo local	2

3) Comportamiento en el infinito:

Cuando $x \rightarrow \pm\infty$, el término dominante $3x^4 \rightarrow +\infty$ (exponente par), por lo que $f(x) \rightarrow +\infty$. La función está **acotada inferiormente** y posee un mínimo global.

4) Mínimo global:

Comparando los mínimos locales: $f(-2) = -25 < f(1) = 2$. Por lo tanto, el **mínimo global** se alcanza en $x = -2$ con valor -25 .

Esto confirma que $x_0 = 0$ es la peor elección posible de punto inicial, ya que coincide con el único máximo local de la función.

c) Sea la función $g : \mathbb{R}^2 \rightarrow \mathbb{R}$ dada por

$$g(x, y) = x^2 + y^3 + 3xy + 1$$

c.i) [0.5 puntos] Aplíquese el método sobre $g(x, y)$ con $x_0 = (-1, 1)$, $\gamma = 0.01$, `tol=1e-12`, `maxit=1e5`.

```
In [13]: # c.i) x0=(-1, 1), gamma=0.01
resultado = descenso_gradiente(grad_g, x0=[-1.0, 1.0], gamma=0.01, tol=1e-12)
x, y = resultado
print(f"Aproximación: (x, y) = ({x:.6f}, {y:.6f})")
print(f"g(x, y) = {x**2 + y**3 + 3*x*y + 1:.6f}")
```

Aproximación: (x, y) = (-2.250000, 1.500000)
 $g(x, y) = -0.687500$

c.ii) [0.5 puntos] ¿Qué ocurre si ahora partimos de $x_0 = (0, 0)$? ¿Se obtiene un resultado deseable?

```
In [14]: # c.ii) x0=(0, 0), gamma=0.01
resultado = descenso_gradiente(grad_g, x0=[0.0, 0.0], gamma=0.01, tol=1e-12)
x, y = resultado
print(f"Aproximación: (x, y) = ({x:.6f}, {y:.6f})")
print(f"Gradiente: {grad_g(resultado)}")
print(f"g(x, y) = {x**2 + y**3 + 3*x*y + 1:.6f}")
```

Aproximación: (x, y) = (0.000000, 0.000000)
Gradiente: [0. 0.]
 $g(x, y) = 1.000000$

R/

No, no es un resultado deseable. El método se queda estancado en el punto inicial $(0, 0)$ sin moverse.

Esto se debe a que $(0, 0)$ es un punto crítico de g , donde el gradiente vale exactamente cero:

$$\nabla g(0, 0) = (2x + 3y, 3y^2 + 3x)|_{(0,0)} = (0, 0)$$

Como la actualización es $x = x - \gamma \cdot \nabla g(x)$, si el gradiente es cero el punto nunca cambia y el algoritmo se detiene de inmediato.

Además, $(0, 0)$ no es un mínimo sino un **punto silla**: el determinante del Hessiano en ese punto es negativo, lo que indica que la función crece en una dirección y decrece en otra. Por tanto, converger a este punto es indeseable, ya que no corresponde a un mínimo de la función. (El análisis completo se desarrolla en el apartado siguiente.)

c.iii) [0.5 puntos] Realícese el estudio analítico de la función y utilícese para explicar y contrastar los resultados obtenidos en los dos apartados anteriores.

R/

Sea

$$g(x, y) = x^2 + y^3 + 3xy + 1$$

1) Gradiente

$$\frac{\partial g}{\partial x} = 2x + 3y \quad \frac{\partial g}{\partial y} = 3y^2 + 3x$$

$$\nabla g(x, y) = (2x + 3y, 3y^2 + 3x)$$

2) Puntos críticos ($\nabla g = 0$)

De $2x + 3y = 0$ se obtiene $x = -\frac{3y}{2}$. Sustituyendo en $3y^2 + 3x = 0$:

$$3y^2 - \frac{9y}{2} = 0 \implies 3y \left(y - \frac{3}{2} \right) = 0 \implies y = 0 \text{ ó } y = \frac{3}{2}$$

- Para $y = 0$: $x = 0 \Rightarrow (0, 0)$
- Para $y = \frac{3}{2}$: $x = -\frac{9}{4} \Rightarrow \left(-\frac{9}{4}, \frac{3}{2} \right)$

3) Matriz Hessiana

$$\frac{\partial^2 g}{\partial x^2} = 2, \quad \frac{\partial^2 g}{\partial x \partial y} = 3, \quad \frac{\partial^2 g}{\partial y^2} = 6y$$

$$H_g(x, y) = \begin{pmatrix} 2 & 3 \\ 3 & 6y \end{pmatrix}$$

En $(0, 0)$:

$$H_g(0, 0) = \begin{pmatrix} 2 & 3 \\ 3 & 0 \end{pmatrix}, \quad \det = (2)(0) - (3)(3) = -9 < 0 \implies \text{punto silla}$$

En $\left(-\frac{9}{4}, \frac{3}{2}\right)$:

$$H_g = \begin{pmatrix} 2 & 3 \\ 3 & 9 \end{pmatrix}, \quad \det = (2)(9) - (3)(3) = 9 > 0, \quad g_{xx} = 2 > 0 \implies \text{mínimo local}$$

Valor en el mínimo:

$$g\left(-\frac{9}{4}, \frac{3}{2}\right) = \frac{81}{16} + \frac{27}{8} - \frac{81}{8} + 1 = -\frac{11}{16} \approx -0.6875$$

4) Contraste con los apartados anteriores

Apartado	Punto inicial	Resultado	Tipo de punto
c.i	$(-1, 1)$	$\left(-\frac{9}{4}, \frac{3}{2}\right)$	Mínimo local ✓
c.ii	$(0, 0)$	$(0, 0)$	Punto silla ✗

- En **c.i**, partiendo de $(-1, 1)$, el método se aleja de la región del punto silla y converge correctamente al **mínimo local** en $\left(-\frac{9}{4}, \frac{3}{2}\right)$, con $g \approx -0.6875$. Este es el resultado deseable.
- En **c.ii**, partiendo de $(0, 0)$, el algoritmo arranca exactamente sobre el **punto silla**, donde $\nabla g = 0$. Como la actualización depende del gradiente, el método no se mueve y queda atrapado en un punto que **no es un mínimo**.

Esto ilustra dos limitaciones del descenso de gradiente: depende fuertemente del **punto inicial**, y puede **estancarse en puntos críticos que no son mínimos** (sillas o máximos) cuando el gradiente se anula en ellos.

R/

El comportamiento observado depende fundamentalmente de la tasa de aprendizaje empleada en el descenso de gradiente. Una tasa excesivamente alta puede generar inestabilidad y divergencia, impidiendo que el algoritmo alcance el punto óptimo; por el contrario, una tasa igual a cero anula cualquier actualización de los parámetros, dejando al modelo estancado sin evolución alguna.

En el caso de la función $g(x,y)=x^2+y^3+3xy+1$, el análisis analítico revela que el punto crítico se localiza en el intervalo $(-94,32)(-49,23)$, coincidiendo con la solución obtenida mediante la implementación algorítmica, lo que confirma la validez del método programado.

No obstante, al variar la tasa de aprendizaje, se ratifican los escenarios ya señalados: con valores extremos — como 0 o magnitudes muy grandes — el algoritmo no logra converger adecuadamente. Además, se evidencia que el éxito de la convergencia también depende del punto de partida, ya que distintas inicializaciones pueden llevar a resultados divergentes o a mínimos locales, subrayando la sensibilidad del método tanto al hiperparámetro como a la condición inicial.